

Cluster-Startprobleme, Bibliothekskonflikte und Out-of-Memory-Diagnose

Section 6 · Troubleshooting, Monitoring and Optimization · Quelle: Databricks-Dokumentation (Compute Troubleshooting)

Die vorangegangenen Kapitel dieser Section haben sich mit Performance-Problemen befasst, die auftreten, *während* ein Job läuft (Skew, Shuffle, Spill). Genauso wichtig für die Prüfung ist die zweite Kategorie von Problemen: Ein Cluster startet gar nicht erst richtig, oder er stürzt kurz nach dem Start ab – oft aus ganz anderen Gründen als reiner Datenmenge.

1. Cluster-Startfehler diagnostizieren

Wenn ein Cluster nicht in den Status *Running* gelangt, lohnt sich zuerst ein Blick auf die Fehlerkategorie, die Databricks im Cluster-Event-Log meldet:

- **Timeout beim Hochfahren** – der Treiberknoten meldet sich nicht innerhalb des konfigurierten Zeitfensters zurück. Häufige Ursache: Init-Skripte oder das Herunterladen zusätzlicher Bibliotheken/Treiber (z. B. Hive-Metastore-JARs) dauert zu lange.
- **Cloud-Provider-Limits** – das Kontingent an verfügbaren Instanzen/vCPUs in der Region ist erschöpft, oder die angeforderte Instanzfamilie ist in der Verfügbarkeitszone gerade nicht verfügbar (häufig bei Spot-Instances).
- **Init-Skript-Fehler** – ein global oder clusterspezifisch hinterlegtes Init-Skript bricht mit einem Fehler ab (z. B. fehlende Datei, Berechtigungsproblem, defekter Download-Link) und verhindert dadurch den kompletten Start.
- **Nicht erreichbarer Treiber** – Netzwerk-/Firewall-Konfiguration verhindert die Kommunikation zwischen Control Plane und Compute Plane (häufig bei benutzerdefinierten VPC-/VNet-Einstellungen).

Der erste Diagnoseschritt ist immer derselbe: das **Cluster-Event-Log** (Reiter „Event Log“ in der Cluster-Detailansicht) sowie die **Treiber-Logs** (`stderr` / `stdout` unter „Driver Logs“) öffnen – dort steht in der Regel die konkrete Fehlermeldung, nicht nur der allgemeine Status „Terminated“.

2. Bibliothekskonflikte erkennen und beheben

Wird eine Bibliothek clusterweit installiert (z. B. über die UI, eine `requirements.txt` oder ein Init-Skript) und ist mit einer bereits im Databricks Runtime enthaltenen Version inkompatibel, kann das den Treiber bereits beim Start zum Absturz bringen – typisches Beispiel sind Binärkonflikte zwischen einer manuell installierten NumPy/Pandas-Version und den C-Erweiterungen, die fest im Runtime-Image verankert sind.

```
# Systematisches Vorgehen bei Verdacht auf Bibliothekskonflikt:
# 1. Cluster OHNE automatische Bibliotheksinstallation starten -> startet er normal?
# 2. Falls ja: Bibliotheken einzeln nacheinander hinzufügen, nach jedem Schritt neu
    starten
# 3. Sobald der Fehler wieder auftritt, ist die zuletzt hinzugefügte Bibliothek die
    Ursache
# 4. Lösung meist: exakte, im Runtime mitgelieferte Version verwenden statt "neueste"
    Version
```

Bei clusterweit über ein Init-Skript installierten Bibliotheken lohnt sich zusätzlich ein Blick in die Treiber-Logs auf Meldungen wie `ImportError` oder `version conflict` direkt nach dem Start – diese verraten meist exakt, welche zwei Paketversionen kollidieren.

3. Out-of-Memory-Fehler diagnostizieren

Ein **OOM-Fehler** (Out of Memory) kann sowohl den Treiber als auch einzelne Executor betreffen – die Ursache und die Abhilfe unterscheiden sich:

Symptom	Typische Ursache	Abhilfe
Driver OOM	<code>collect()</code> , <code>toPandas()</code> oder große Broadcast-Variablen holen zu viele Daten auf den Treiberknoten	Aggregation/Filterung vor dem Sammeln, größeren Treiber wählen, Broadcast-Threshold senken
Executor OOM	Data Skew, zu wenige Shuffle-Partitionen, teure Operationen wie <code>explode()</code> oder <code>collect_set()</code>	siehe Kapitel zu Skew/Shuffle/Spill dieser Section; mehr RAM pro Core bereitstellen
Wiederholte OOM-Kills während des Cluster-Starts	Zu kleine Instanztypen für die installierten Bibliotheken/Init-Skripte selbst (z. B. speicherintensive ML-Bibliotheken)	größeren Treiber-/Worker-Instanztyp wählen

Die Spark-UI-Reiter **Executors** (Speicherauslastung je Executor) und **Stages** (Spill-Metriken, siehe vorheriges Kapitel) sind auch hier die erste Anlaufstelle, bevor die Cluster-Größe vorschnell erhöht wird – oft liegt die eigentliche Ursache im Code oder im Datenlayout, nicht in zu wenig Hardware.

Ergänzt aus der offiziellen Databricks-Dokumentation: Databricks nennt als häufigste Ursachen für gescheiterte Cluster-Starts Timeouts, fehlerhafte globale oder clusterspezifische Init-Skripte, zu viele gleichzeitig installierte Bibliotheken, Cloud-Provider-Limits sowie nicht erreichbare Instanzen. Als erster Diagnoseschritt wird empfohlen, den Cluster testweise komplett ohne automatische Bibliotheksinstallation zu starten, um festzustellen, ob überhaupt ein Bibliothekskonflikt vorliegt, bevor einzelne Bibliotheken schrittweise wieder ergänzt werden.

Quelle: [Troubleshoot compute issues – Databricks-Dokumentation](#)